

ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ
KHOA CÔNG NGHỆ THÔNG TIN

TÀI LIỆU BỔ SUNG THỰC HÀNH
CASE STUDY VÀ HANDS-ON
PHÁT TRIỂN ỨNG DỤNG WEB MÃ NGUỒN MỞ

Tài liệu bổ sung cho giáo trình
Học qua tình huống – thực hành – sản phẩm

Biên soạn bổ sung
2026

MỞ ĐẦU

Tài liệu này là phần bổ sung thực hành cho giáo trình “Phát triển ứng dụng web mã nguồn mở”. Nội dung chuyển kiến thức theo từng chương thành các tình huống gần với một dự án Web thực tế, giúp người học kết nối lý thuyết với thiết kế, lập trình, kiểm thử và trình bày sản phẩm.

Cấu trúc khuyến nghị: Câu hỏi củng cố → Bài tập cơ bản → Hands-on → Case Study → Bài tập mở rộng → Dự án tổng hợp.

1. Mục tiêu

- Kết nối kiến thức lý thuyết với bài toán ứng dụng.
- Rèn kỹ năng phân tích yêu cầu, thiết kế, lập trình và kiểm thử.
- Tạo sản phẩm có thể phát triển liên tục qua nhiều chương.
- Hình thành thói quen sử dụng README, dữ liệu mẫu và kiểm thử.
- Tăng mức độ vận dụng ở Laravel, RESTful API và bảo mật.

2. Quy ước chung

Thành phần	Quy ước
Ngôn ngữ	PHP, HTML5, CSS3, JavaScript
CSDL	MySQL
Môi trường	XAMPP/WAMP hoặc PHP tương đương
Framework	Laravel theo phiên bản được quy định trong học phần
API	RESTful API, JSON, HTTP
Kiểm thử API	Postman
Bảo mật	Prepared Statement, Output Encoding, CSRF, Session, Cookie, Password Hashing, Authentication/Authorization

Ghi chú: Phiên bản phần mềm có thể thay đổi theo phòng máy; ưu tiên phiên bản phù hợp với PHP/Laravel được giảng viên quy định.

CASE STUDY 1 – TRIỂN KHAI MÔI TRƯỜNG WEB CỤC BỘ

Chủ đề: Từ máy tính cá nhân đến một website PHP chạy trên Web Server cục bộ.

1.1. Bối cảnh

Một nhóm sinh viên cần xây dựng website giới thiệu câu lạc bộ học thuật. Nhóm chưa cần thuê hosting; toàn bộ phát triển và kiểm thử thực hiện trên máy cá nhân. Nhiệm vụ đầu tiên là thiết lập môi trường Web, tạo website và quan sát quá trình trình duyệt gửi yêu cầu đến Web Server.

1.2. Mục tiêu

- Cài đặt và kiểm tra Apache, PHP và MySQL/MariaDB.
- Tạo cấu trúc website PHP đơn giản.
- Phân biệt Browser, Web Server, PHP và CSDL.
- Quan sát Request URL, HTTP Method, Status Code và Content-Type bằng DevTools.
- Biết tạo README mô tả cách chạy bài thực hành.

1.3. Yêu cầu Case Study

- Cài XAMPP/WAMP hoặc môi trường PHP tương đương.
- Khởi động Apache; khởi động MySQL/MariaDB nếu sử dụng.
- Tạo thư mục `web-lab` trong web root.
- Tạo `css`, `js`, `images`.
- Tạo `index.php` hiển thị tên nhóm, học phần, thời gian hiện tại và mô tả câu lạc bộ.
- Tạo CSS riêng.
- Mở `http://localhost/web-lab/`.
- Mở DevTools → Network và ghi nhận thông tin request đến `index.php`.

1.4. Hands-on 1 – Tạo website giới thiệu nhóm

```
web-lab/  
├── index.php  
├── css/  
│   └── style.css  
├── js/  
│   └── app.js  
├── images/  
└── README.md
```

Ví dụ `index.php`:

```
<?php  
date_default_timezone_set('Asia/Ho_Chi_Minh');  
?>  
<!DOCTYPE html>  
<html lang="vi">  
<head>  
  <meta charset="UTF-8">  
  <title>Web Lab</title>  
  <link rel="stylesheet" href="css/style.css">  
</head>
```

```
<body>
  <main class="container">
    <h1>Nhóm phát triển Web</h1>
    <p>Học phần: Phát triển ứng dụng web mã nguồn mở</p>
    <p>Thời gian: <?= date('d/m/Y H:i:s') ?></p>
  </main>
</body>
</html>
```

1.5. Hands-on 2 – Phân tích HTTP bằng DevTools

- Mở trang `index.php`.
- Mở Developer Tools → Network.
- Tải lại trang.
- Chọn request đến `index.php`.
- Ghi Request URL, Method, Status Code và Content-Type.
- Thực hiện tương tự với CSS và JavaScript.
- Giải thích vì sao trình duyệt phải gửi nhiều request.

1.6. Sản phẩm cần nộp

- Thư mục `web-lab`.
- Ảnh website đang chạy.
- Ảnh Network trong DevTools.
- README.md.
- Mô tả luồng Browser → Web Server → PHP → Response.

1.7. Mở rộng

- Thêm `about.php`, `contact.php`.
- Tạo menu dùng chung.
- Thêm JavaScript cho nút tương tác.
- Tạo form liên hệ để chuẩn bị cho GET/POST.

CASE STUDY 2 – WEBSITE QUẢN LÝ THÔNG TIN THÀNH VIÊN

Chủ đề: Form, biến, điều kiện, hàm, GET/POST, Cookie, Session và xử lý dữ liệu PHP.

2.1. Bối cảnh

Một câu lạc bộ, nhóm học tập hoặc tổ chức cần xây dựng một website nhỏ để quản lý thông tin thành viên. Hệ thống cho phép nhập và cập nhật thông tin cơ bản, tìm kiếm thành viên, lưu tùy chọn hiển thị và mô phỏng trạng thái đăng nhập.

Bài toán không phụ thuộc vào quy định riêng của từng trường. Người học có thể áp dụng cho câu lạc bộ, nhóm dự án, tổ chức sinh viên hoặc một đơn vị bất kỳ.

2.2. Thông tin quản lý

Thành phần	Mô tả
Họ và tên	Tên thành viên
Email	Địa chỉ email
Số điện thoại	Thông tin liên hệ
Vai trò	Thành viên, trưởng nhóm, cộng tác viên...
Sở thích	Một hoặc nhiều lĩnh vực quan tâm
Ghi chú	Thông tin bổ sung

Hệ thống không yêu cầu một cấu trúc thành viên cố định. Giảng viên có thể điều chỉnh các trường dữ liệu tùy theo tình huống thực hành.

2.3. Hands-on 1 – Form nhập thông tin thành viên

- Tạo form nhập: Họ và tên, Email, Số điện thoại, Vai trò, Sở thích và Ghi chú.
- Sử dụng phương thức POST để gửi dữ liệu.
- Kiểm tra các trường bắt buộc.
- Kiểm tra định dạng email.
- Kiểm tra độ dài phù hợp của dữ liệu.
- Sử dụng biến PHP để tiếp nhận và xử lý dữ liệu.
- Viết hàm kiểm tra dữ liệu.
- Hiện thị thông báo khi dữ liệu hợp lệ hoặc không hợp lệ.

Ví dụ:

```
function kiểmTraEmail($email) {  
    return filter_var($email, FILTER_VALIDATE_EMAIL);  
}  
  
if ($_SERVER['REQUEST_METHOD'] === 'POST') {  
    $name = trim($_POST['name'] ?? '');  
    $email = trim($_POST['email'] ?? '');  
  
    if ($name === '') {  
        echo "Vui lòng nhập họ và tên.";
```

```

    } elseif (!kiemTraEmail($email)) {
        echo "Email không hợp lệ.";
    } else {
        echo "Thông tin hợp lệ.";
    }
}

```

2.4. Hands-on 2 – Tìm kiếm và hiển thị thông tin

- Hiển thị danh sách thành viên mẫu.
- Tìm kiếm theo họ tên.
- Lọc theo vai trò.
- Sử dụng GET để truyền từ khóa tìm kiếm.
- Hiển thị thông báo khi không tìm thấy dữ liệu.
- Phân biệt dữ liệu nhận từ GET và POST.

Ví dụ: members.php?keyword=Nguyen

Người học cần giải thích:

- Vì sao chức năng tìm kiếm có thể sử dụng GET?
- Khi nào nên sử dụng POST?
- Dữ liệu từ người dùng cần được kiểm tra như thế nào?

2.5. Hands-on 3 – Cookie và Session

Cookie:

- Lưu tên hiển thị của thành viên.
- Cho phép ghi nhớ tùy chọn giao diện hoặc thông tin hiển thị.
- Cho phép xóa Cookie.

Session:

- Mô phỏng trạng thái đăng nhập.
- Lưu thông tin người dùng hiện tại.
- Kiểm tra Session trước khi truy cập trang quản lý.
- Xây dựng chức năng đăng xuất.

Ví dụ:

```
session_start();
```

```

$_SESSION['user'] = [
    'name' => 'Nguyen Van A',
    'role' => 'Member'
];

```

2.6. Case Study mở rộng

- Thêm chức năng sửa thông tin thành viên.
- Thêm chức năng xóa thành viên.
- Tìm kiếm theo nhiều tiêu chí.
- Lọc thành viên theo vai trò.
- Phân trang danh sách.
- Lưu dữ liệu vào file JSON hoặc MySQL.
- Thêm chức năng đăng nhập/đăng xuất.
- Phân biệt quyền của Member và Admin.
- Thêm chức năng thay đổi thông tin cá nhân.
- Thiết kế giao diện Responsive bằng Bootstrap.

2.7. Sản phẩm

- Mã nguồn.
- Dữ liệu mẫu.
- Ảnh giao diện.
- Ảnh kết quả kiểm thử.
- README hướng dẫn chạy chương trình.

Sơ đồ xử lý:

- **Form → GET / POST → PHP xử lý dữ liệu → Cookie / Session → Response**

CASE STUDY 3 – HỆ THỐNG QUẢN LÝ SẢN PHẨM

Chủ đề: PHP + MySQL, CRUD, tìm kiếm, lọc và truy vấn an toàn.

3.1. Bối cảnh

Một cửa hàng cần hệ thống quản lý sản phẩm nội bộ. Quản trị viên quản lý danh mục, sản phẩm, tìm kiếm và lọc.

3.2. Thiết kế dữ liệu

Bảng	Trường chính	Ý nghĩa
users	id, username, password	Tài khoản quản trị
categories	id, name, created_at	Danh mục
products	id, category_id, name, price, quantity, description, created_at	Sản phẩm

3.3. Yêu cầu chức năng

- CRUD danh mục.
- CRUD sản phẩm.
- Xem chi tiết.
- Tìm kiếm theo tên.
- Lọc theo danh mục.
- Tính tổng giá trị tồn kho.
- Validation.
- Prepared Statements.
- Không nối trực tiếp dữ liệu người dùng vào SQL.

3.4. Hands-on 1 – CRUD sản phẩm

- Tạo CSDL và bảng.
- Tạo kết nối MySQL.
- Trang danh sách.
- Form thêm.
- Sửa.
- Xóa.
- Chi tiết.
- Tìm kiếm/lọc.
- Thông báo thành công/thất bại.

3.5. Hands-on 2 – Prepared Statement

```
$stmt = $conn->prepare(  
    "SELECT id, name, price, quantity  
    FROM products  
    WHERE name LIKE ?"  
);
```

```
$keyword = "%" . $keyword . "%";  
$stmt->bind_param("s", $keyword);  
$stmt->execute();  
$result = $stmt->get_result();
```

Ghi chú: Mục tiêu là nhận diện sự khác nhau giữa truy vấn nối chuỗi và Prepared Statement.

3.6. Hands-on 3 – Báo cáo tồn kho

```
SELECT SUM(price * quantity) AS inventory_value  
FROM products;
```

Mở rộng theo từng danh mục nếu muốn tăng mức độ vận dụng.

3.7. Tiêu chí hoàn thành

- CRUD đúng.
- Validation.
- Prepared Statement ở chức năng có dữ liệu người dùng.
- Thông báo kết quả.
- Dữ liệu mẫu.
- README và file SQL.

CASE STUDY 4 – HỆ THỐNG QUẢN LÝ KHÓA HỌC THEO MVC

Chủ đề: Model – View – Controller, Router và phân tách trách nhiệm.

4.1. Bối cảnh

Từ hệ thống CRUD Chương 3, nhóm tổ chức lại mã nguồn theo MVC để tách giao diện, nghiệp vụ và truy cập dữ liệu.

4.2. Thành phần

Thành phần	Nội dung
Category	Quản lý danh mục khóa học
Course	Quản lý khóa học
Controller	Tiếp nhận request và điều phối
Model	Truy vấn dữ liệu
View	Hiển thị danh sách/form/chi tiết

4.3. Cấu trúc thư mục

```
app/  
├── controllers/  
│   ├── CourseController.php  
│   └── CategoryController.php  
├── models/  
│   ├── Course.php  
│   └── Category.php  
├── views/  
│   ├── course/  
│   └── category/  
├── config/  
├── routes/  
├── public/  
└── index.php
```

4.4. Hands-on 1 – Luồng MVC

```
Request  
↓  
Router  
↓  
CourseController  
↓  
Course Model  
↓  
MySQL  
↓  
CourseController  
↓  
View  
↓  
HTTP Response
```

4.5. Hands-on 2 – CRUD

- Thiết kế `categories`, `courses`.
- Tạo Model.
- Tạo Controller.
- Khai báo Route.
- Tạo View danh sách.
- Tạo View thêm/sửa.
- Xóa.
- Chi tiết.
- Tìm kiếm.
- Vẽ lại luồng request.

4.6. Câu hỏi phân tích

- Nếu SQL nằm trong View thì khó khăn gì?
- Nếu Controller chứa toàn bộ HTML thì bảo trì ra sao?
- Model và Controller khác nhau thế nào?
- Request `/course/detail?id=10` đi qua thành phần nào?

4.7. Sản phẩm

- Mã nguồn MVC.
- Sơ đồ thư mục.
- Sơ đồ Request → Router → Controller → Model → DB → View.
- Ảnh CRUD.
- README.

CASE STUDY 5 – HỆ THỐNG QUẢN LÝ SỰ KIỆN VỚI LARAVEL

Chủ đề: Laravel MVC, Route, Controller, Model, Migration, Eloquent, Blade, Validation, CSRF và Session.

5.1. Bối cảnh

Một trường đại học cần website quản lý hoạt động/sự kiện sinh viên. Admin tạo sự kiện; sinh viên xem và đăng ký.

5.2. Thiết kế dữ liệu

Bảng	Một số trường	Quan hệ
users	id, name, email, password	Người dùng
categories	id, name	Danh mục
events	id, category_id, title, description, event_date, location, capacity	Thuộc danh mục
registrations	id, event_id, user_id, registered_at	Người dùng đăng ký

5.3. Chức năng

Vai trò	Chức năng
Admin	Đăng nhập; CRUD danh mục/sự kiện; xem đăng ký
Student/Member	Xem sự kiện; chi tiết; đăng ký; hủy

5.4. Hands-on 1 – Khởi tạo Laravel

- Tạo project.
- Cấu hình `.env`.
- Tạo application key.
- Tạo migration.
- Chạy migration.
- Tạo Model và quan hệ Eloquent.
- Tạo Controller.
- Khai báo Route.
- Tạo Blade Layout/View.

5.5. Hands-on 2 – CRUD sự kiện

- Danh sách.
- Thêm.
- Validation.
- Lưu.
- Sửa.
- Xóa.

- Thông báo.
- Tìm kiếm/lọc.
- Phân trang nếu cần.

5.6. Hands-on 3 – Đăng ký sự kiện

- Xem chi tiết.
- Yêu cầu đăng nhập.
- Đăng ký.
- Không đăng ký trùng.
- Không vượt sức chứa.
- Hủy đăng ký.
- Hiện thị sự kiện đã đăng ký.

5.7. Yêu cầu bảo mật

- Form POST có CSRF protection.
- Validation dữ liệu.
- Kiểm soát quyền Admin.
- Không đưa dữ liệu nhạy cảm ra giao diện.
- Không lưu mật khẩu dạng văn bản thuần.
- Không công khai `.env`.

5.8. Mở rộng

- Tìm kiếm.
- Lọc danh mục.
- Lọc ngày.
- Số chỗ còn lại.
- Xuất danh sách đăng ký.
- Dashboard thống kê.

5.9. Deliverables

- Source code Laravel.
- Migration/dữ liệu mẫu.
- README.
- Ảnh giao diện.
- Sơ đồ CSDL.
- Danh sách route.
- Danh sách kiểm thử.

CASE STUDY 6 – XÂY DỰNG RESTFUL API QUẢN LÝ KHÓA HỌC

Chủ đề: REST, JSON, HTTP Status Code, API Resource, Sanctum, Postman và Fetch.

6.1. Bối cảnh

Hệ thống quản lý khóa học cần cung cấp dữ liệu cho Frontend độc lập. Backend trả JSON thay vì HTML.

6.2. Endpoint

Method	Endpoint	Chức năng	Kết quả
GET	/api/courses	Danh sách	200
GET	/api/courses/{id}	Chi tiết	200/404
POST	/api/courses	Tạo mới	201
PUT	/api/courses/{id}	Cập nhật	200/404
DELETE	/api/courses/{id}	Xóa	204/404

6.3. Hands-on 1 – CRUD API

- Kích hoạt hệ thống API.
- Tạo API Controller.
- Tạo API Resource.
- Khai báo route.
- CRUD.
- JSON nhất quán.
- Xử lý 404.
- Kiểm thử Postman.

6.4. Bộ kiểm thử Postman

Test	Request	Expected
T1	GET /api/courses	200 + JSON
T2	GET /api/courses/1	200 + JSON
T3	GET /api/courses/99999	404
T4	POST /api/courses	201
T5	PUT /api/courses/1	200
T6	DELETE /api/courses/1	204

6.5. Hands-on 2 – Fetch

```
async function loadCourses() {  
  const response = await fetch('/api/courses');  
  const data = await response.json();  
}
```

```

    if (!response.ok) {
        throw new Error('Không thể tải dữ liệu');
    }

    renderCourses(data);
}

```

Tạo `courses.html`, gọi API và hiển thị danh sách. Frontend không chứa SQL hoặc logic truy cập MySQL.

6.6. Hands-on 3 – API có xác thực

- Tạo endpoint đăng nhập.
- Xác thực người dùng.
- Cấp token theo cơ chế Laravel được cấu hình.
- Gọi API riêng tư không token.
- Gọi lại với token hợp lệ.
- Kiểm thử tạo/xóa khi đã xác thực.
- Giải thích Session/Cookie và Bearer Token.

6.7. Câu hỏi phân tích

- Vì sao API trả JSON?
- Ý nghĩa 200, 201, 204, 400, 401, 403, 404?
- Stateless có ý nghĩa gì?
- API dùng Cookie cần xem xét CSRF như thế nào?
- Vì sao không đưa mật khẩu vào response?

6.8. Sản phẩm

- Project API.
- Postman Collection.
- Kết quả kiểm thử.
- Trang HTML/JS.
- Tài liệu endpoint.

CASE STUDY 7 – PHÒNG THÍ NGHIỆM BẢO MẬT ỨNG DỤNG WEB

Chủ đề: SQL Injection, XSS, CSRF, Password Hashing, Session, Cookie, Authentication, Authorization và API Security.

Ghi chú: Chỉ thực hiện kiểm thử trên ứng dụng do sinh viên tự xây dựng hoặc môi trường lab được giảng viên cho phép. Không kiểm thử hệ thống bên thứ ba khi chưa được cấp quyền.

7.1. Bối cảnh

Một ứng dụng quản lý người dùng được xây dựng trong môi trường cục bộ. Sinh viên tạo phiên bản có lỗi có kiểm soát, quan sát hiện tượng, sau đó sửa mã nguồn và kiểm thử lại.

7.2. Mục tiêu

- Nhận diện nguyên nhân SQL Injection.
- Phân biệt dữ liệu người dùng và mã lệnh.
- Hiểu XSS và Output Encoding.
- Hiểu CSRF trong ứng dụng dùng Cookie/Session.
- Biết lưu mật khẩu bằng cơ chế hashing dành cho mật khẩu.
- Phân biệt Authentication và Authorization.
- Kiểm tra các điểm truy cập API cần bảo vệ.

7.3. Hands-on 1 – SQL Injection và Prepared Statement

// Chỉ dùng để minh họa trong lab:

```
$sql = "SELECT * FROM users WHERE username = " . $username . "";
```

// Phiên bản bảo vệ:

```
$stmt = $conn->prepare(
    "SELECT * FROM users WHERE username = ?"
);
$stmt->bind_param("s", $username);
```

1. Quan sát cách truy vấn được tạo ở phiên bản minh họa lỗi.
2. Giải thích vì sao input có thể ảnh hưởng cấu trúc truy vấn.
3. Chuyển sang Prepared Statement.
4. Kiểm thử lại.
5. Ghi kết quả.

7.4. Hands-on 2 – XSS và Output Encoding

```
echo htmlspecialchars(
    $comment,
    ENT_QUOTES,
    'UTF-8'
);
```

- Xác định điểm dữ liệu người dùng đi vào giao diện.
- Phân biệt validation và output encoding.
- Áp dụng escaping khi xuất HTML.
- Kiểm thử dữ liệu bình thường và dữ liệu chứa ký tự HTML.

7.5. Hands-on 3 – Authentication an toàn

- Lưu mật khẩu bằng `password\_hash()`.
- Xác thực bằng `password\_verify()`.
- Quản lý đăng nhập bằng Session.
- Cookie nếu sử dụng phải có thuộc tính bảo mật phù hợp.
- Kiểm thử đăng nhập đúng, sai và đăng xuất.

```
$hash = password_hash($password, PASSWORD_DEFAULT);
```

```
if (password_verify($password, $hash)) {  
    // Xác thực thành công  
}
```

7.6. Hands-on 4 – Security Audit

Hạng mục	Vị trí	Nguyên nhân/rủi ro	Biện pháp	Kết quả
SQL Injection	Form/query			
XSS	Trang hiển thị			
CSRF	Form thay đổi dữ liệu			
Password Hashing	Đăng ký/đổi mật khẩu			
Session	Đăng nhập/đăng xuất			
Cookie	Cookie ứng dụng			
Authentication	Endpoint yêu cầu đăng nhập			
Authorization	Admin/Member			
Input Validation	Form/API			
Output Encoding	View			
API Authentication	API riêng tư			

7.7. Hands-on 5 – Kiểm tra CSRF

- Chọn form thay đổi dữ liệu.
- Xác định cơ chế xác thực.
- Kiểm tra CSRF token.
- Thử request thiếu token trong lab.
- Ghi nhận kết quả.
- Giải thích mối liên hệ giữa CSRF, Cookie/Session và cách request được gửi.

7.8. Hands-on 6 – Bảo vệ API

- Xác định API công khai/riêng tư.
- Không token.
- Token không hợp lệ.
- Token hợp lệ.
- Không trả thông tin nhạy cảm.
- Validation POST/PUT.
- Ghi nhận status code.

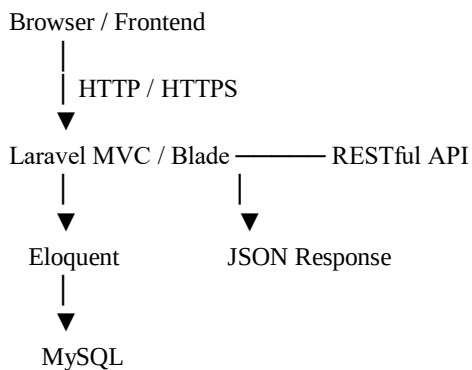
DỰ ÁN TỔNG HỢP – HỆ THỐNG QUẢN LÝ HOẠT ĐỘNG SINH VIÊN

Dự án tổng hợp kết nối kiến thức toàn bộ giáo trình thành một sản phẩm.

1. Chức năng

- Quản lý tài khoản.
- Quản lý khoa/bộ môn.
- Quản lý học phần.
- Quản lý hoạt động/sự kiện.
- Sinh viên đăng ký.
- Admin quản trị.
- RESTful API.
- Authentication.
- Authorization.
- Bảo vệ form và dữ liệu.

2. Kiến trúc đề xuất



3. Ánh xạ kiến thức

Chương	Nội dung áp dụng
1	Môi trường Web, HTTP, Web Server, cấu trúc dự án
2	PHP, Form, GET/POST, Session, Cookie
3	MySQL, CRUD, Prepared Statement
4	MVC, Router, Controller, Model, View
5	Laravel, Migration, Eloquent, Blade, Validation, CSRF
6	RESTful API, JSON, Postman, Fetch, API Authentication
7	SQL Injection, XSS, CSRF, Password Hashing, Session, Cookie, API Security

4. Sản phẩm bắt buộc

```
project/
├── source-code/
├── database/
│   └── database.sql
├── docs/
│   ├── architecture.pdf
│   └── api-documentation.pdf
├── screenshots/
└── README.md
```

5. README tối thiểu

6. Giới thiệu.
7. Thành viên.
8. Công nghệ.
9. Yêu cầu môi trường.
10. Cài đặt.
11. Cấu hình CSDL.
12. Cách chạy.
13. Tài khoản demo.
14. Danh sách API.
15. Biện pháp bảo mật.
16. Chức năng chưa hoàn thiện.

6. Tiêu chí đánh giá đề xuất

Tiêu chí	Tỷ trọng gợi ý
Phân tích yêu cầu và thiết kế	15%
Chức năng nghiệp vụ	25%
Kiến trúc và tổ chức mã nguồn	15%
CSDL và xử lý dữ liệu	10%
RESTful API	10%
Bảo mật	15%
Giao diện	5%
README/demo/thuyết trình	5%

PHỤ LỤC A – RUBRIC ĐÁNH GIÁ

Mức	Mô tả
4 – Thành thạo	Hoàn thành đầy đủ; hiểu nguyên lý; xử lý trường hợp biên; giải thích được lựa chọn kỹ thuật.
3 – Đạt tốt	Hoàn thành phần lớn; còn hạn chế nhỏ nhưng hệ thống hoạt động đúng.
2 – Đạt một phần	Có chức năng cơ bản nhưng còn lỗi hoặc phụ thuộc nhiều vào hướng dẫn.
1 – Chưa đạt	Chưa hoàn thành chức năng chính hoặc chưa giải thích được cách hoạt động.

Câu hỏi vấn đáp gợi ý

- Request đi qua những thành phần nào?
- Dữ liệu nào do người dùng nhập?
- Tại sao dùng GET/POST?
- Vì sao cần Prepared Statement?
- Tại sao không lưu mật khẩu dạng văn bản thuần?
- Authentication và Authorization khác nhau thế nào?
- Vì sao API trả JSON?
- Status code nào phù hợp?
- CSRF phụ thuộc cơ chế xác thực thế nào?
- Nếu yêu cầu nghiệp vụ thay đổi, thành phần nào cần thay đổi?

PHỤ LỤC B – CHECKLIST TRƯỚC KHI NỘP

- ☐ Project chạy được trên máy khác theo README.
- ☐ Có SQL hoặc migration/seed cần thiết.
- ☐ Không đưa mật khẩu/ khóa bí mật/cấu hình nhạy cảm vào mã nguồn công khai.
- ☐ Form quan trọng có validation.
- ☐ Truy vấn có dữ liệu người dùng dùng cơ chế an toàn.
- ☐ Dữ liệu xuất HTML được xử lý phù hợp.
- ☐ Chức năng yêu cầu đăng nhập có Authentication.
- ☐ Chức năng quản trị có Authorization.
- ☐ API có mô tả endpoint/method/request/response.
- ☐ Postman Collection được cung cấp nếu có API.
- ☐ Có ảnh kết quả kiểm thử.
- ☐ README có hướng dẫn cài đặt và chạy.

PHỤ LỤC C – CẤU TRÚC BÀI NỘP

01-source/
02-database/
03-docs/
04-screenshots/
README.md

KẾT LUẬN

Các Case Study và Hands-on được tổ chức theo hướng tăng dần mức độ vận dụng: từ môi trường Web và HTTP, PHP/MySQL, MVC, Laravel, RESTful API đến bảo mật. Dự án tổng hợp kết nối các nội dung thành một sản phẩm có thể sử dụng cho bài tập lớn hoặc đánh giá cuối học phần.

Giảng viên có thể dùng từng Case Study độc lập hoặc yêu cầu nhóm phát triển liên tục một sản phẩm từ Chương 1 đến Chương 7.